Fakulta elektrotechniky a informatiky
Slovenskej technickej univerzity

PVSO

Zadanie 2: Kalibrácia kamery

Michal Balogh
Jana Kampová

16. marca 2026

Obsah

1 Kalibrácia kamery (Task 1) **2**
1.1 Príprava súradníc 2
1.2 Detekcia 2
1.3 Výpočet modelu a geometria kamery 3
1.4 Odstránenie skreslenia (Undistortion) 4
1.5 Analýza presnosti (Mean Reprojection Error) 4
1.6 Získané parametre kamery 4

2 Detekcia geometrických tvarov (Task 2) **6**
2.1 Princíp detekcie 6
2.2 Predspracovanie obrazu 6
2.3 Detekcia polygonálnych tvarov 6
2.4 Detekcia kružníc - Houghova transformácia 7
2.5 Potlačenie duplicít kružníc - NMS 7
2.6 Vizualizácia a ladenie 8
2.7 Bonus: Určenie veľkosti objektov v centimetroch 9

3 Farebný filter (Task 3) **11**
3.1 Farebný priestor HSV 11
3.2 Implementácia 12
3.3 Výsledky filtrácie 12

1 Kalibrácia kamery (Task 1)

Proces kalibrácie slúži na odstránenie optických chýb šošovky. Celý postup prebieha v nasledujúcich krokoch:

1.1 Príprava súradníc

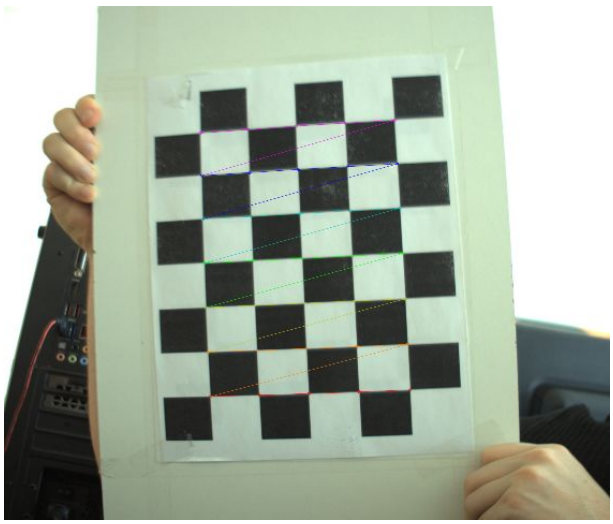
Pri kalibrácii pracujeme s dvoma sadami bodov, ktoré sa algoritmus snaží spárovať:

- **Ideálne súradnice (objpoints)**: Predstavujú „3D body v reálnom svete“ (ideálnu mriežku šachovnice).
  - **objp**: Pole nul pre x, y, z súradnice, kde z ostáva 0 a x, y sa na seba párujú podľa počtu rohov šachovnice.
- **Obrazové súradnice (imgpoints)**: „2D body v rovine obrazu“, teda súradnice z reálneho obrázka šachovnice.

1.2 Detekcia

Pre každú snímku hľadáme vnútorné rohy pomocou **cv2.findChessboardCorners**. Ak sa nájdu všetky rohy (**ret** == **True**), nasleduje :

- **cv2.cornerSubPix**: Táto funkcia hľadá najpresnejšiu súradnicu hrany.
- **Princíp symetrie**: Algoritmus analyzuje okno 11×11 pixelov a zisťuje, kde je matematický gradient najviac symetrický podľa jasu. Tento bod sa považuje za presný matematický stred rohu (+).
- **Vizuálna kontrola**: Pomocou **cv2.drawChessboardCorners** overujeme, či program našiel rohy v správnom poradí (od červenej po modrú farbu).



Obr. 1: Ukážka detekcie vnútorných rohov na šachovnici v našom programe pomocou **cv2.drawChessboardCorners**.

1.3 Výpočet modelu a geometria kamery

Samotný výpočet **cv2.calibrateCamera** porovnáva ideálnu mriežku s pokrivenými bodmi z fotiek.

Vnútorné parametre (Camera Matrix K)

Matica 3×3 matematicky prepojuje 3D priestor s 2D fotkou.

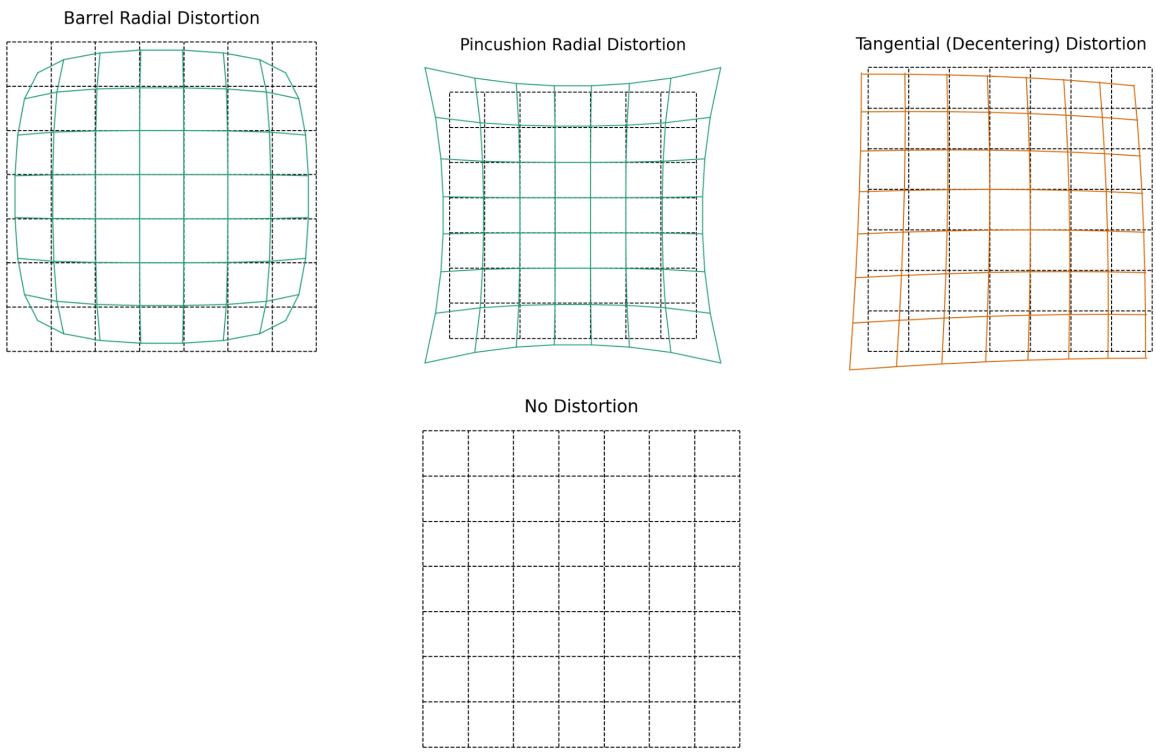
$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

V matici sú kľúčové parametre f_x, f_y (ohnisková vzdialenosť) a c_x, c_y (optický stred).

Model skreslenia (dist)

Reálne šošovky mávajú geometrické deformácie, ktoré v kóde popisujeme koeficientami $(k_1, k_2, p_1, p_2, k_3)$. Rozlišujeme nasledujúce typy:

- **Radiálne skreslenie (k_n)**: Spôsobené tvarom šošovky. Spôsobuje ohýbanie priamok smerom od stredu alebo k stredu obrazu.
- **Tangenciálne skreslenie (p_n)**: Vzniká pri výrobnej nepresnosti, kedy šošovka nie je dokonale rovnobežná so snímačom (Decentering distortion).



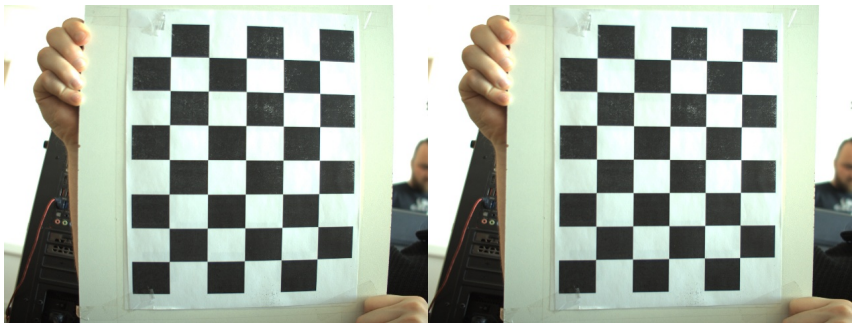
Obr. 2: Zľava: radiálne skreslenie typu Barrel a Pincushion, tangenciálne skreslenie, ideálna mriežka bez skreslenia

1.4 Odstránenie skreslenia (Undistortion)

Pre narovnanie obrazu používame dve metódy definované v **UndistortMethod**:

1. **Metóda CROP**:
   Počíta sa pre každý obrazok. Algoritmus vezme súradnicu rovného pixelu, vypočíta, kde by bol v pokrivenom obraze, a pomocou **interpolácie** (priemer zo 4 susedných pixelov) mu priradi farbu.

2. **Metóda REMAPP**:
   Najskôr vytvorí mapu rozdielov pre každý jeden pixel. Táto mapa sa vytvorí len raz na začiatku a potom sa už len aplikuje na každý nasnímaný obrázok, čo je dobre využiteľné práve pre spracovanie videa v reálnom čase.



Obr. 3: Porovnanie obrazu pred a po kalibrácii kamery. Vľavo pôvodný obraz, vpravo narovnaný obraz po odstránení skreslenia.

Optimalizácia a ROI

Pomocou **cv2.getOptimalNewCameraMatrix** a parametra **alpha=1** (ponechanie všetkých pixelov) získavame **ROI** (Region of Interest).
ROI sú súradnice čistého obdĺžnika bez čiernych rohov $[x, y, rw, rh]$. Vďaka tomu môžeme urobiť finálny orez (**dst[y:y+rh, x:x+rw]**), čím zahodíme prázdne čierne rohy, ktoré vznikli narovnaním.

1.5 Analýza presnosti (Mean Reprojection Error)

Spätná kontrola prebieha tak, že vezmeme 3D body ideálnej šachovnice a cez našu novú maticu ich premietneme naspäť na fotku.

- Používame normu **NORM_L2** (Pytagorova veta), ktorá vypočíta priamu vzdialenosť medzi vypočítaným a skutočným bodom.

- Dosiahnutý výsledok v pixeloch určuje kvalitu: $0.1 - 0.5$ px je veľmi dobrý výsledok pre bežnú kameru.

1.6 Získané parametre kamery

Na základe spracovania sady kalibračných obrázkov sme získali nasledujúce parametre :

- **Matica kamery K:**

$$K = \begin{bmatrix} 3756.5840 & 0 & 1256.2995 \\ 0 & 3750.7221 & 982.8654 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Koeficienty skreslenia:** $[-0.4522, 0.5788, -0.0006, 0.0024, 0.0750]$
- **Mean reprojection error:** 0.1101 px

Parametre boli uložené do súboru `camera_calib.npz`.

2 Detekcia geometrických tvarov (Task 2)

2.1 Princíp detekcie

Program detekuje geometrické tvary v reálnom čase z kamery alebo zo statických obrázkov. Využíva pritom dva prístupy:

- **Kontúrová analýza**: Pre polygonálne tvary (trojuholník, štvorec, obdĺžnik, päťuholník, šesťuholník).
- **Houghova transformácia**: Pre detekciu kružníc.

2.2 Predspracovanie obrazu

Pred samotnou detekciou prebieha niekoľko krokov predspracovania:

1. **Undistortion**: Odstránenie skreslenia šošovky pomocou kalibračných parametrov z Task 1.
2. **Prevod na odtiene šedej**: Konverzia BGR (webkamera) alebo BGRA (Ximea) obrazu na grayscale.
3. **Gaussovo rozmazanie**: Redukcia šumu pred detekciou hrán aj pred Houghovou transformáciou.
4. **Cannyho detektor hrán**: Nájdenie hrán, prahy sa ladia pomocou interaktívnych sliderov.

2.3 Detekcia polygonálnych tvarov

Nájdenie kontúr

Funkcia **cv2.findContours** s príznakom **RETR_EXTERNAL** vráti len vonkajšie kontúry objektov, čím sa vyhneme vnoreným kontúram vo vnútri tvarov.

Aproximácia a klasifikácia

Pri testovaní sa ukázalo, že samotná aproximácia pôvodnej kontúry je citlivá na perspektívu, šum a drobné nerovnosti hrany. Preto pred aproximáciou najskôr počítame **konvexný obal** kontúry (**convex hull**), ktorý odstraňuje malé lokálne zárezy a stabilizuje tvar objektu.

Následne sa konvexný obal aproximuje polygónom pomocou **cv2.approxPolyDP**. Tolerancia aproximácie nie je pevná, ale je riadená hyperparametrom **poly_eps**, pričom v implementácii používame dolnú hranicu 1.8 % dĺžky obvodu. Počet vrcholov aproximovaného polygónu slúži ako základná informácia pre klasifikáciu:

Pre rozlíšenie štvorca a obdĺžnika sa počíta pomer strán (*aspect ratio*) ohraničujúceho obdĺžnika:

$$\text{aspect} = \frac{b_w}{b_h}$$

Ak $0.95 < \text{aspect} < 1.05$, tvar je klasifikovaný ako **Štvorec**, inak ako **Obdĺžnik**.

Počet vrcholov	Tvar
3	Trojuholník (Triangle)
4	Štvorec alebo Obdĺžnik
5	Päťuholník (Pentagon)
6	Šesťuholník (Hexagon)

Tabuľka 1: Klasifikácia tvarov podľa počtu vrcholov polygónu

Okrem toho používame aj pomocnú mieru **rectangularity**, ktorá porovnáva plochu kontúry s plochou minimálneho otočeného obdĺžnika (**minAreaRect**). Ak aproximácia vplyvom perspektívy alebo šumu vytvorí 5 až 6 vrcholov, ale hodnota rectangularity ostane vysoká, objekt sa stále interpretuje ako **štvorec** alebo **obdĺžnik**. Tento krok výrazne znižuje chybné označenie obdĺžnikových tvarov ako pentagón alebo hexagón.

Pred samotnou klasifikáciou sa zároveň odfiltrujú príliš malé kontúry pomocou hyperparametra **Min area**, aby sa potlačili falošné detekcie vzniknuté šumom v obraze.

Ťažisko každého tvaru sa vypočíta pomocou momentov kontúry (inšpirácia z [6]), kde m_{pq} sú momenty kontúry:

$$c_x = \frac{m_{10}}{m_{00}}, \quad c_y = \frac{m_{01}}{m_{00}}$$

2.4 Detekcia kružníc - Houghova transformácia

Kružnice nemajú ostré vrcholy, preto aproximácia polygónom nefunguje tak spoľahlivo. Namiesto toho používame **cv2.HoughCircles** s metódou **HOUGH_GRADIENT**, ktorá pracuje priamo s gradientmi obrazu. Houghova transformácia v aktuálnej implementácii používa ten istý rozmazaný obraz ako Cannyho detektor. Tým pádom sa polygonálna detekcia aj detekcia kružníc správajú konzistentne a reagujú na rovnaký hyperparameter rozmazania.

Parametre Houghovej transformácie

- **dp**: Pomer rozlíšenia akumulátora voči vstupnému obrazu.
- **minDist**: Minimálna vzdialenosť medzi stredmi susedných kružníc.
- **param1**: Horný prah Cannyho detektora použitého interne v Houghovej transformácii. Tento parameter je zámerne naviazaný na rovnaký slider ako Cannyho detektor použitý pri kontúrach.
- **param2**: Prah akumulátora - nižšia hodnota znamená viac falošných detekcií.
- **minRadius, maxRadius**: Očakávaný rozsah polomerov kružníc.

2.5 Potlačenie duplicít kružníc - NMS

Houghova transformácia môže pre jednu kružnicu vrátiť viacero kandidátov s podobnými stredmi. Vlastná funkcia **_nms_circles** implementuje *Non-Maximum Suppression*:

1. Kandidáti sa zoradia zostupne podľa polomeru.

2. Pre každý neoznačený kandidát i sa potlačia všetci kandidáti j, ktorých stred leží vo vzdialenosti menšej ako $0.8 \cdot \max(r_i, r_j)$ od stredu i.

```
order = np.argsort(-circles[:, 2])  # zoradenie podla polomeru
circles = circles[order]

for i in range(len(circles)):
    if suppressed[i]:
        continue
    cx1, cy1, r1 = circles[i]
    for j in range(i + 1, len(circles)):
        if suppressed[j]:
            continue
        cx2, cy2, r2 = circles[j]
        d = np.hypot(cx1 - cx2, cy1 - cy2)
        if d < dist_thresh_ratio * max(r1, r2):
            suppressed[j] = True
```

Listing 1: Core NMS algoritmu pre kružnice

2.6 Vizualizácia a ladenie

Program zobrazuje okrem výsledného obrazu aj pomocné debug okno so štyrmi pohľadmi:

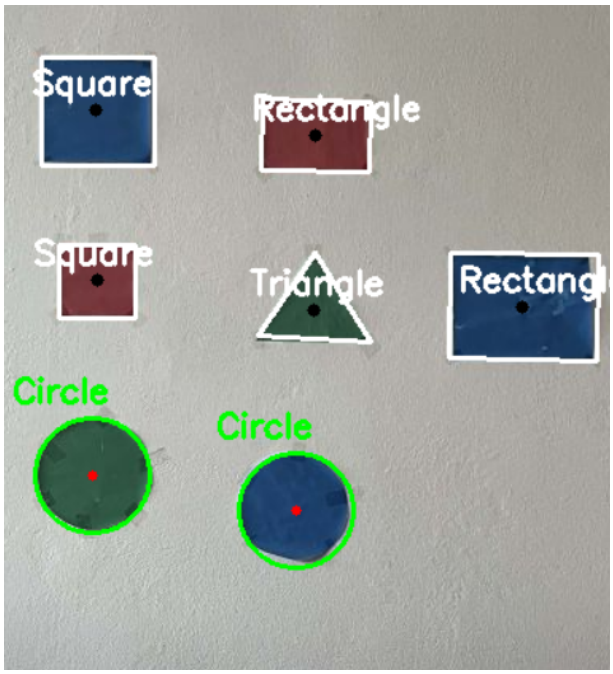
- Grayscale obraz
- Rozmazaný (Gaussian) obraz
- Výsledok Cannyho detektora hrán
- Nájdené kontúry

Detekované polygonálne tvary sú označené bielym obrysom s názvom tvaru. Kružnice sú označené zeleným kruhom, červenou bodkou v strede a nápisom *Circle*.

Súčasťou programu je aj samostatné okno **Settings** s interaktívnymi slidermi. Používateľ môže počas behu meniť hyperparametre:

- vzdialenosť kamery od objektov Z,
- horný prah Cannyho detektora,
- veľkosť Gaussovho rozmazania,
- minimálnu plochu kontúry,
- parameter aproximácie **poly_eps**,
- parametre Houghovej transformácie (dp, $minDist$, $param2$, rozsah polomerov),
- parameter *Non-Maximum Suppression* pre potláčanie duplicitných kružníc.

Vďaka týmto sliderom je možné algoritmus dolaďovať priamo nad živým obrazom bez potreby meniť zdrojový kód a opakovane spúšťať program.



Obr. 4: Detekované geometrické tvary.

2.7 Bonus: Určenie veľkosti objektov v centimetroch

Princíp výpočtu

Na prepočet pixelovej veľkosti objektu na reálne rozmery využívame model perspektívnej kamery. Zo vzťahu podobných trojuholníkov platí:

$$\frac{\text{object_size}}{Z} = \frac{\text{image_size}}{f_x}$$

Z toho vyjadríme reálnu veľkosť objektu:

$$\text{object_size} = \frac{\text{image_size} \cdot Z}{f_x}$$

kde f_x je ohnisková vzdialenosť v pixeloch z kalibrovanej matice kamery K a Z je vzdialenosť roviny objektov od kamery v centimetroch. Pre vertikálne rozmery sa analogicky použije f_y.

Implementácia

Vzdialenosť Z je inicializovaná konštantou **CAMERA_DISTANCE_CM**, ale počas behu ju možno meniť aj interaktívne sliderom v okne **Settings**. Výpočet prebieha vo funkcii **px_to_cm**:

```
CAMERA_DISTANCE_CM = 30.0  # fyzicka vzdialenost kamery od objektov [cm]

def px_to_cm(pixels, axis="x"):
    if _mtx is None:
        return None
    f = _mtx[0, 0] if axis == "x" else _mtx[1, 1]
    return pixels * camera_distance_cm / f
```

Listing 2: Prepočet pixelov na centimetre pomocou kalibračnej matice

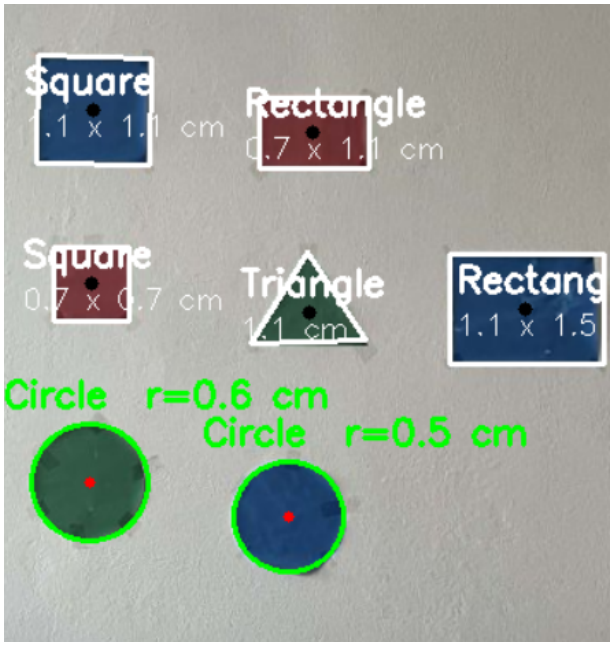
Zobrazované rozmery

Pre každý detekovaný tvar sa vypočíta a zobrazí príslušná veličina:

Tvar	**Zobrazená veličina**
Kruh (Circle)	Polomer r v cm
Štvorec (Square)	Dĺžka strany v cm
Obdĺžnik (Rectangle)	Šírka $\times$ Výška v cm
Trojuholník (Triangle)	Najdlhšia strana v cm
Pentagon / Hexagon	Priemerná dĺžka strany v cm

Tabuľka 2: Zobrazované rozmery pre jednotlivé typy tvarov

Ak kalibračný súbor **camera_calib.npz** nie je dostupný, funkcia **px_to_cm** vráti `None` a rozmery sa nezobrazujú.

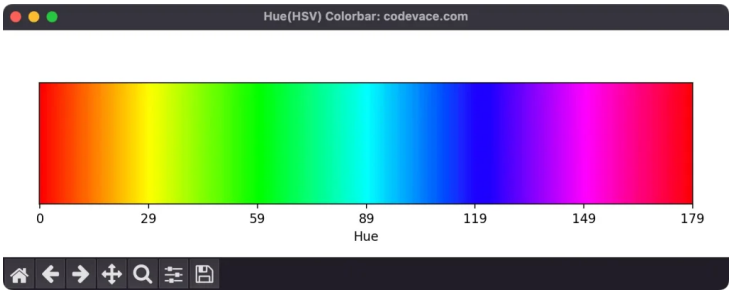


Obr. 5: Detekované geometrické tvary a ich veľkosti.

3 Farebný filter (Task 3)

3.1 Farebný priestor HSV

Pre detekciu farieb sme použili model **HSV**. Hlavným dôvodom je, že na rozdiel od BGR, HSV oddeľuje jasovú zložku (Value) od farebnej (Hue). To nám umožňuje nastaviť filter, ktorý je odolný voči tieňom a zmenám osvetlenia v miestnosti.



Obr. 6: HSV farebný model používaný aj v knižnici OpenCV.

Škálovanie hodnôt v OpenCV

Na rozdiel od bežného matematického modelu, OpenCV upravuje rozsahy hodnôt pre potreby spracovania obrazu:

- **Odtieň (H):** Štandardný rozsah 0°–360° je vydelený dvoma, čím vzniká rozsah **0–180**.
- **Sýtosť (S) a Jas (V):** Pôvodný rozsah 0–100% je prepočítaný na hodnoty **0–255**.

V kóde sme pre jednotlivé farby určili nasledujúce hodnoty.
Červená farba vyžaduje spojenie dvoch masiek lebo v HSV kruhu sa nachádza na oboch koncoch intervalu (pri 0° aj 180°).

Farba	**Hue (Odtieň)**	**Saturation (Sýtosť)**	**Value (Jas)**
Červená (1)	0 – 10	100 – 255	100 – 255
Červená (2)	170 – 180	100 – 255	100 – 255
Zelená	50 – 70	50 – 255	50 – 255
Modrá	110 – 130	50 – 255	50 – 255

Tabuľka 3: HSV rozsahy

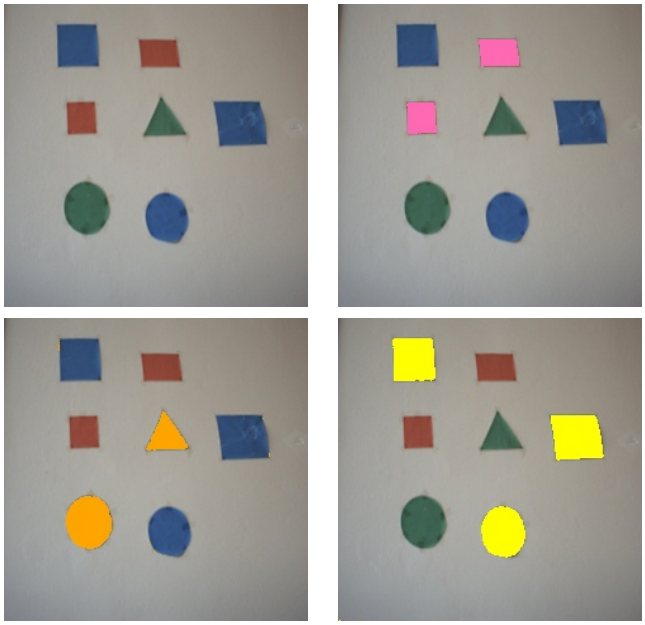
3.2 Implementácia

```
# Nastavenie cielovej farby pre modry filter

if state == SwapState.BLUE:

    # Hladame modru (100-140), ignorujeme tiene (V>50) a bledo-sive
   odtiene (S>50)

    maska = cv2.inRange(hsv, (100, 50, 50), (140, 255, 255))
    nova_farba_bgr = (0, 255, 255) # BGR kod pre zltu

# Aplikacia podla typu pripojeneho zariadenia
if num_channels == 3:  # Notebook (BGR)
    camera = ()
    image[maska > 0] = nova_farba_bgr + camera

elif num_channels == 4: # Ximea (BGRA)
    camera = (255,)
    image[maska > 0] = nova_farba_bgr + camera
```

Listing 3: Implementácia farebného filtra pre 3 a 4-kanálové kamery

3.3 Výsledky filtrácie

Na nasledujúcom obrázku je zobrazená funkčnosť filtra pre jednotlivé farebné kanály. Pôvodný obraz je porovnaný s výsledkom po aplikácii HSV masky, kde detekovaná červená farba je nahradená ružovou, detekovaná modrá je nahradená žltou a detekovaná zelená je nahradená oranžovou.



Obr. 7: Porovnanie: pôvodný obrázok (hore vľavo), červená nahradená ružovou (hore vpravo), zelená nahradená oranžovou (dole vľavo), modrá nahradená žltou (dole vpravo).

Literatúra

[1] Tangram Vision Blog. *Exploring Distortion and Distortion Models.* Dostupné na: https://www.tangramvision.com/blog/camera-modeling-exploring-distortion-and-distortion-models-part-iii

[2] OpenCV Documentation. *Changing Colorspaces.* Dostupné na: https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html

[3] LearnOpenCV. *Color Spaces in OpenCV.* Dostupné na: https://learnopencv.com/color-spaces-in-opencv-cpp-python/

[4] Camera Calibration, OpenCV Documentation. Dostupné na: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

[5] Hough Transform, OpenCV Documentation. *Hough Circle Transform.* Dostupné na: https://docs.opencv.org/4.x/da/d53/tutorial_py_houghcircles.html

[6] How to Detect Shapes in Images in Python using OpenCV? *Image Moments.* Dostupné na: https://www.geeksforgeeks.org/python/how-to-detect-shapes-in-images-in-python-using-opencv/

[7] Online HSV Converter a kalkulačka pre OpenCV rozsahy. Dostupné na: https://vgyp.github.io/OpenCV-HSV-Color-Picker/